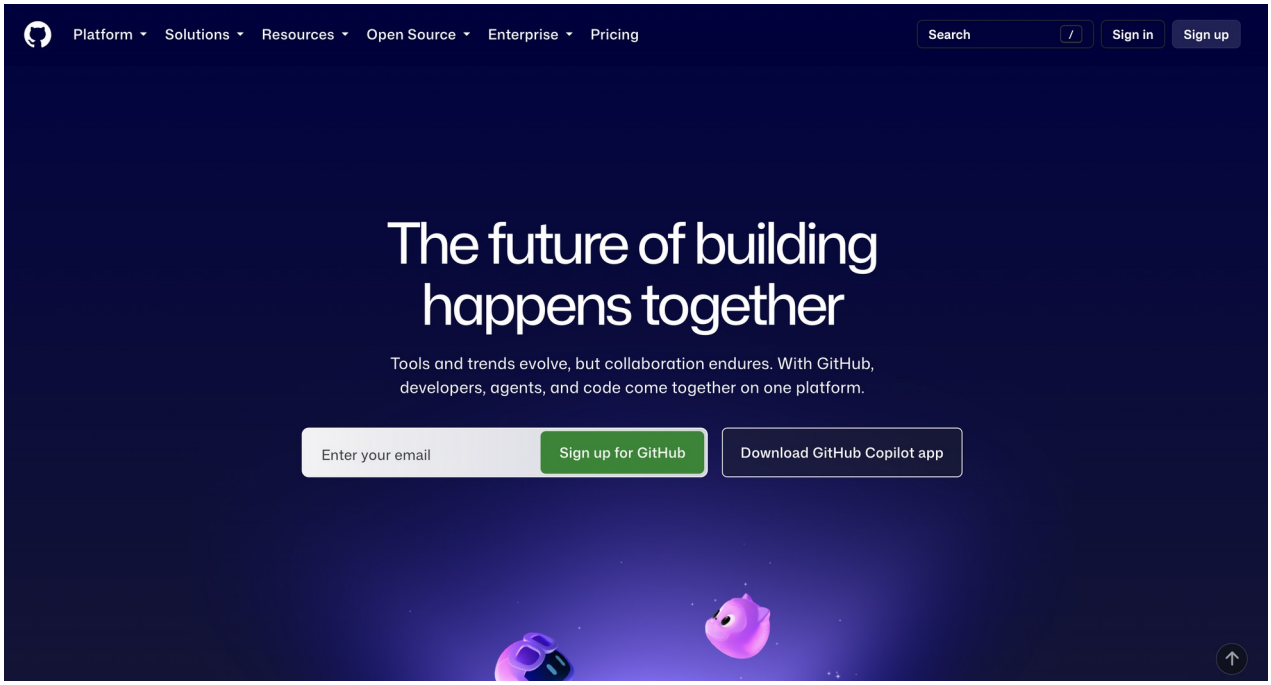
GitHub × Codex：從 Fork 範本到完成自己的作品



Part I | Git 與 GitHub 的基本原理

1. Git 、 GitHub

Git 是版本控制工具，協助開發者在本機追蹤檔案變化、建立歷史紀錄、切換不同開發分支。
GitHub 是以 Git 為核心的線上程式碼託管與協作平台，提供 Repository、Issues、Pull Request、Code Review 與 GitHub Pages 等功能。

名詞	作用	可以這樣理解
Git	管理程式版本與變更歷史	專案的時光機
GitHub	線上存放與協作專案	專案的雲端工作區
Repository（Repo）	專案檔案與版本歷史的集合	一個有版本紀錄的專案資料夾
Commit	儲存一個有說明的變更紀錄	一次有原因的進度存檔
Branch	從既有開發歷史延伸出的工作線	不影響主要版本的實驗區
Pull Request（PR）	提出合併某組變更的請求	請同伴檢視與討論這次修改

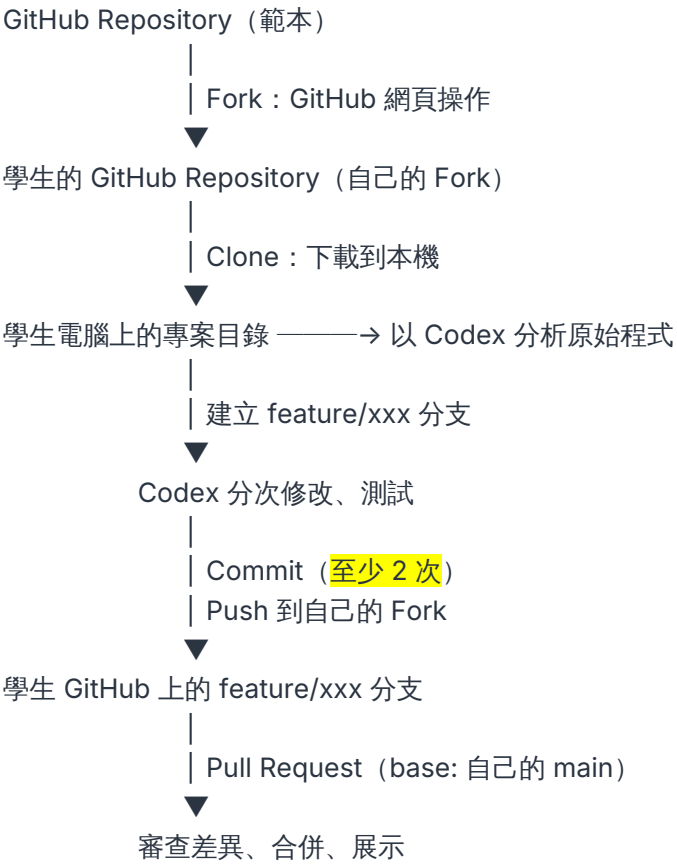
題目 1： 把 index.html 複製到 USB 隨身碟，也可以備份，為什麼程式開發還需要 Git？

作答：USB 備份只是另一份檔案，看不出改了哪裡、為什麼改，也很難回到某一個版本。Git 會把每次修改記成有說明的 commit，可以用 git diff 比較差異、退回舊版本，也能開分支試做新功能而不影響 main，多人還能各自在分支上開發再合併。這次作業我把新功能放在 feature/my-improvement 分支，和 main 的原作品分開，用 PR 看兩者的差異；出問題時也能從 commit 紀錄找出是哪一次改動造成的再修正。

2. Fork、Clone、Branch 差在哪裡？

- **Fork**： 在 GitHub 上建立一份與原始 Repository 有關聯、但可獨立管理的 Repo，通常位於自己的帳號。
- **Clone：** 把 GitHub 上的 Repo 複製到自己的電腦，形成可直接編輯的本機工作目錄。
- **Branch：** 在同一個 Repo 的版本歷史中建立一條工作線，用於開發新功能或修正問題。
- **Commit：** 將已選取的變更記錄為一個版本節點；它不是把檔案傳到 GitHub。
- **Push**： 將本機的 Commit 傳送至遠端 Repo（本課程為自己的 Fork）。
- **Pull**： 從遠端取得並整合變更；與 **Pull Request** 是不同概念。
- **Pull Request**： 比較兩條分支的差異，請求審查並選擇是否合併。

本課堂工作流程：


GitHub Repository（範本）
Fork：GitHub 網頁操作
學生的 GitHub Repository（自己的 Fork）
Clone：下載到本機
學生電腦上的專案目錄 ———→ 以 Codex 分析原始程式
建立 feature/xxx 分支
Codex 分次修改、測試
Commit（至少 2 次）
Push 到自己的 Fork
學生 GitHub 上的 feature/xxx 分支
Pull Request（base: 自己的 main）
審查差異、合併、展示

3. GitHub 最常見的三個工作位置

工作位置	內容
原始 Repo	作品範本
個人的遠端 Fork	GitHub 上自己的專案
本機 Clone	電腦上讓 Codex 操作的程式

4. Git 與 AI 協作時的角色分工

- **學生：** 決定目標、讀懂需求、授權操作、檢查差異與驗證成果。
- **Codex：** 協助閱讀程式、提出修改方案、修改檔案、執行測試、分析錯誤。
- **Git**： 追蹤版本、保留 Commit、協助比較及還原變更。
- **GitHub：** 提供遠端備份、協作、PR 與成果分享。

Codex 能執行許多程式操作，但學生須對每項修改、命令與交付成果負責。操作前應確認工作目錄，尤其是安裝依賴、刪除檔案、Push、合併或發布時。

Part II | 上機前準備

5. 課前檢查

- ☐ 已註冊且登入 **GitHub**：https://github.com/
- ☐ 不把 API Key、密碼、帳號憑證或個資提交到公開 Repo。

Codex 使用方式：

在 Codex 中開啟本機專案資料夾，於對話框要求 Codex 讀取、修改及測試程式。

6. 範本作品

選擇一個範本，做修改、執行、demo。

GitHub 範本網址	
1 https://github.com/educatres/microscope	https://educatres.github.io/microscope/
2 https://github.com/educatres/red-blood-cell-heart-adventure	https://educatres.github.io/red-blood-cell-heart-adventure/
3 https://github.com/educatres/gravity-slingshot-voyager-game	https://educatres.github.io/gravity-slingshot-voyager-game/
4 https://github.com/educatres/finboardgame	https://educatres.github.io/finboardgame/
5 https://github.com/kisaraki/classroom-nibbles-tzk	https://kisaraki.github.io/classroom-nibbles-tzk/
6 https://github.com/educatres/simpleKnowledgeChatbot	https://educatres.github.io/simpleKnowledgeChatbot/
7 https://github.com/Sonville/suzanne-paper-plane-lab	https://sonville.github.io/suzanne-paper-plane-lab/
8 https://github.com/educatres/chopstick-bridge-lab	https://educatres.github.io/chopstick-bridge-lab/
9 https://github.com/educatres/infinite-whiteboard	https://educatres.github.io/infinite-whiteboard/
10 https://github.com/educatres/food-digestion-simulator	https://educatres.github.io/food-digestion-simulator/

我選擇的範本： finboardgame（經濟棋局，範本編號 4）
原始 Repo 網址： https://github.com/educatres/finboardgame
原始作品的主要用途（自己描述，至少 30 字）：
這是一款 2–8 人在同一台電腦輪流玩的財經桌遊。玩家各扮演一種企業，用兩個 0–6 的拉霸決定前進格數，在棋盤上抽全球、個人、產業事件，並用聘僱、裁員、借款等決策經營公司，同時觀察通膨、失業率與 AI 聯準會的升降息，目標是讓淨資產達標，藉此學習總體經濟與企業財務的關係。

Part III｜上機實作：Fork → Codex → Push

任務 1｜Fork 到自己的 GitHub

目標： 建立自己的遠端 Repository，不直接更動教師提供的範本。

操作：

1. 登入 GitHub，開啟所選範本的 Repository 頁面。
2. 先觀察 README、License、檔案結構、最近的 Commit。
3. 點選右上角 **Fork**，Owner 選擇自己的帳號。
4. 確認 Repository 名稱，點選 **Create fork**。
5. 成功後核對網址是 https://github.com/你的帳號/專案名稱，並可看到 Fork 來源。

紀錄：
我的 GitHub 帳號：soymilk0211
我的 Fork 網址：https://github.com/soymilk0211/bigdata-week7

證據 1： 截圖自己的 Fork 首頁，畫面需看得到帳號名稱、Repo 名稱、Fork 來源。

紀錄：GitHub 查詢結果：soymilk0211/bigdata-week7 是 Fork（fork: true）、公開的 Repo，預設分支是 main，已開啟 GitHub Pages。Fork 首頁的畫面要登入 GitHub 才能擷取，請本人補上截圖。

檢核問題： Fork 之後，老師的原始 Repo 會因為你修改自己的 Fork 而立即改變嗎？為什麼？

作答：不會。Fork 是複製到自己帳號底下的獨立 Repo，我修改自己的 Fork 不會影響老師的原始 Repo，除非之後送出 Pull Request 且老師同意合併。

任務 2｜Clone 到本機

目標： 讓 Codex 有一個可以實際讀寫的專案資料夾。

操作： 在自己的 Fork 頁面點選 **Code → HTTPS**，複製網址，接著在終端機執行：

```
git clone https://github.com/YOUR-USERNAME/YOUR-REPO.git
cd YOUR-REPO
git remote -v
git status
```

注意： YOUR-USERNAME、YOUR-REPO 是要自己替換的佔位文字。Clone 完成後，origin 應指向自己的 **Fork**，而不是原作者的 Repo。

若希望用 Codex 協助 Git 操作，可貼入以下提示詞：

我已經 Fork 一份 GitHub 專案。請先確認目前資料夾是否是 Git Repository，執行或引導我執行 git remote -v、git status，解釋 origin 指向哪裡。任何可能改寫檔案或操作遠端 GitHub 的命令，請先說明並等我確認。

紀錄：
本機專案路徑：C:\Users\csie2-\Desktop\week7\bigdata-week7
origin 指向的網址：https://github.com/soymilk0211/bigdata-week7.git

證據 **2**： 截圖終端機的 git remote -v 與 git status（或 Codex 呈現的執行結果）。



實際執行結果：origin 指向自己的 Fork（github.com/soymilk0211/bigdata-week7），目前在 feature/my-improvement 分支，和 GitHub 上的版本一致，沒有未提交的修改。

任務 3｜用 Codex 理解原始作品

目標： 在修改程式之前，先弄清楚程式功能與執行方法。

操作： 在 Codex 開啟本機 Repo，貼入下列提示詞：

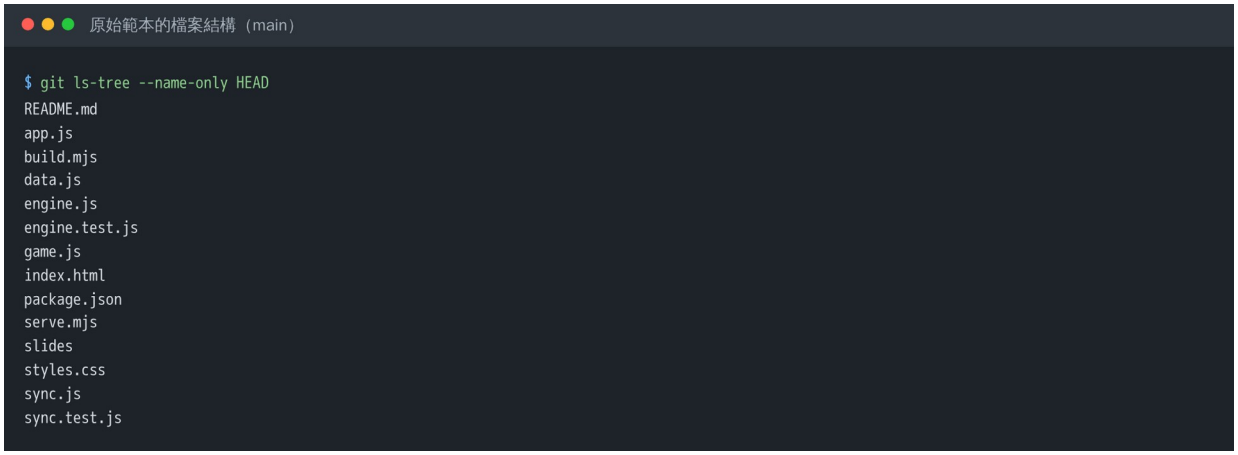
請先閱讀本專案 README、主要程式檔、設定檔與目錄結構，暫時不要修改任何檔案。請用適合大學部學生的方式整理：① 這個作品想解決什麼問題；② 程式進入點及主要模組；③ 使用的程式語言與框架；④ 如何安裝和執行；⑤ 現有功能；⑥ 可改良的 5 個方向。每項結論請指出對應的檔案名稱，無法確認的地方請明確標示。

學生需自行確認 Codex 的分析，不要直接複製答案。 若 Codex 說明與實際程式不一致，追問它引用哪個檔案及哪一段程式。

紀錄：

問題	我的紀錄
原作品功能	多人輪流的財經桌遊「經濟棋局」：兩個 0–6 的拉霸決定前進格數，在 24 格棋盤上抽全球、個人、產業事件。 玩家扮演 10 種企業，可聘僱裁員、借還款；季末結算後由 10 條規則的 AI 聯準會調整利率，並有總體經濟儀表板、歷史紀錄、存檔匯出匯入與多頁籤同步 （index.html、engine.js、data.js、app.js）。
可改良的 5 個方向	1: 把歷史紀錄匯出成 CSV，方便用 Excel 分析 （app.js、engine.js） 2: 對局結束時列出所有玩家的淨資產排名，而不只顯示勝出者 （app.js 結束面板） 3: 額外借款尚未產生每季利息，README 也有說明（engine.js 的 financials） 4: 手機版排版與可近用性（styles.css、index.html） 5: 開局可調整事件頻率，或新增更多事件卡（data.js）

證據 3： 截圖 Codex 說明專案架構的對話，以及專案目錄結構。



原始範本的檔案：index.html（頁面）、app.js（介面）、engine.js（拉霸、回合、事件與結算的規則引擎）、data.js（10 種企業、事件卡與聯準會規則）、sync.js（多頁籤同步）、styles.css、build.mjs（把來源檔合併成 game.js）、serve.mjs（本機伺服器）、engine.test.js／sync.test.js（測試）。Codex 說明架構的對話請本人從 Codex 補上截圖。

任務 4｜先執行原作品，建立比較基準

目標： 確認原作品本來就能跑，並保留「修改前」畫面。

操作：

1. 依 README 的指示安裝依賴並執行專案；如果文件不完整，請 Codex **先分析需要哪些步驟，再執行**。
2. 在瀏覽器或程式視窗確認原有功能可以使用。
3. 記錄遇到的警告、錯誤，判斷是環境問題還是程式問題。
4. 擷取原作品的畫面；如為非 GUI 專案，可擷取 CLI 輸出或測試結果。

範例提問：

請根據這個 Repository 的 README 和設定檔判斷最適合的啟動方式。先列出預計使用的指令與可能風險，得到我的確認後再執行。若啟動失敗，請先找出原因，不要一次修改大量程式。

證據 4： 修改前作品截圖 + 原始版本如何使用的簡短紀錄。





修改前（main 分支的原作品）：直接開啟 index.html（或執行 node serve.mjs 後打開 http://127.0.0.1:4173/），首頁按「設定玩家，開始遊戲」，選 2–8 位玩家和企業後開始。所有人在同一台電腦輪流按「啟動拉霸」、抽事件卡、聘僱或借款經營，全部人完成後結算一季。只能在同一台裝置輪流操作，不能用自己的手機加入。

任務 5｜建立獨立開發分支與 AGENTS.md

目標： 避免直接在 main 亂改，也讓 Codex 遵守專案規範。

操作：

```
git switch -c feature/my-improvement
git branch --show-current
git status
```

接著請 Codex 協助建立或檢查專案根目錄的 AGENTS.md。如果原作品已經有 AGENTS.md，先閱讀，不要直接覆蓋。

建議的 **AGENTS.md** 內容（依專案修改）：

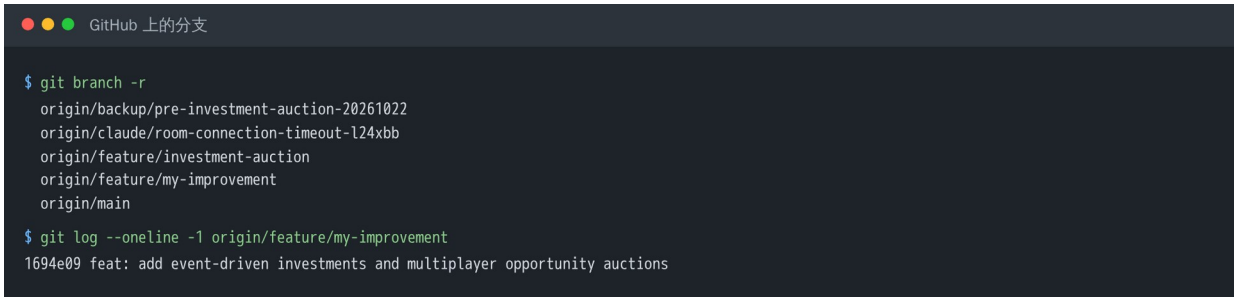
```
# AGENTS.md

## 專案目的
維持原作品的主要功能，透過小步驟完成課堂指定改良。

## 修改規則
- 修改前先指出涉及的檔案及計畫。
- 優先保留原有結構，避免無關的大規模重構。
- 新增功能需有可操作的驗證方式。
- 修改 UI 時保留基本的行動裝置可用性。
- 不建立或提交 API Key、密碼、個資或 .env 等敏感檔案。
- 每完成一個功能，摘要修改範圍與測試結果。

## 交付規則
- 更新 README，說明新增功能、啟動方法及限制。
- 在 Commit 之前先列出修改的檔案，不自行 Push 或合併。
```

證據 5： 新 github 分支的截圖。



GitHub 上的分支：main（原作品，留給 PR 比較用）、feature/my-improvement（本次改良，GitHub Pages 從這裡部署）。另外有組員開發投資與競標用的 feature/investment-auction 和合併前的備份分支，以及 AI 工具工作用的 claude/room-connection-timeout-l24xbb。

任務 6｜至少兩項改良方案

目標： 學生做需求設計，Codex 做方案分析。

建議條件：

- **改良 A（功能性）：** 新增一項能實際操作的功能，例如搜尋、排序、資料匯出、表單驗證、圖表切換等。
- **改良 B（易用性或品質）：** 改善介面、RWD、錯誤處理、可近用性、效能、測試等。
- 文件： 更新 README，清楚介紹自己的修改。
- 不應僅更改標題、作者姓名、顏色或 README 就當作兩項完整改良。

Codex 提示詞：

參考目前程式架構，請提出 3 組適合 60 分鐘內完成的「功能改良＋易用性改良」組合。每組都列出：使用者需求、修改檔案、預估風險、測試方式與驗收標準。不要開始實作，等我選定方案。

請填寫：

項目	學生自己的決定
改良 A：想解決的問題	原作品只能在同一台電腦輪流玩，大家要擠在同一個螢幕前，不能用自己的手機操作自己的公司。
改良 A：預期成果	首頁分成「一台裝置輪流玩」和「多人連線」兩種模式，多人連線不會動到單機存檔。多人連線：房主建立 5 碼房間代碼或邀請連結，其他人用自己的手機加入同一局。進房後每個人自己輸入名字、選一家企業（別人選走的不能選），房主隨時可以開始；開始後每台裝置就對應自己選的企業並鎖定，輪到誰就只有那台能操作。連線用 WebRTC 點對點（PeerJS），不用另外架伺服器，可直接放在 GitHub Pages。
改良 B：想解決的問題	連線失敗時，訪客畫面一直停在「正在連線…」，沒有任何提示；房主手機鎖螢幕後，房間可能被拆掉或換代碼；玩家區一次列出所有人的完整資料太擁擠，也看不出哪位是自己。
改良 B：預期成果	連不上時最多約 15 秒就顯示原因，並建議改用手機熱點；房主暫時離線時保留房間和代碼，回來後自動恢復；找不到房間時說明可能原因；玩家區只顯示自己的完整資料並標「你」，其他人改看「淨資產排行」（名次、公司、淨資產）。
驗證兩項功能的方法	① node --test 單元測試 34 個；② 用 Playwright 在 Chromium 實際連線（真的 PeerJS＋本機配對伺服器），測試選企業大廳、座位鎖定、連線逾時、斷線恢復等情境；③ 兩支手機在不同網路實機連線遊玩；④ 用手機寬度 375px 檢查版面沒有跑掉。

證據 6： 截圖 Codex 提供的候選方案，以及你選擇方案的原因（30 字以上）。

選擇方案的原因：原作品只能大家擠在同一台電腦輪流玩，但上課時每個人都有手機，所以改良 A 選「多人連線」，讓每個人用自己的手機加入同一局。實際用手機測試時，發現連不上時畫面會一直停在「正在連線…」、玩家資料太擠也看不出哪個是自己，所以改良 B 選連線錯誤處理和畫面精簡。（Codex 提供的候選方案請本人從 Codex 補上截圖。）

任務 7 | Codex 分次完成改良

目標： 以小步驟完成需求，不把全部工作塞進一個含糊的提示詞。

第一次提示（改良 **A**）：

我選擇的改良 A 是：「新增「多人連線」：房主建立 5 碼房間代碼和邀請連結，其他人用自己的手機加入同一局；進房後每個人輸入名字、選一家企業（同一家只能一個人選），房主隨時可以開始，開始後每台裝置只能操作自己的企業」。請先確認要修改的檔案，保留原有功能，再實作這一項改良。完成後請說明改動了哪些檔案、如何操作新功能，以及應做哪些測試。不要同時實作改良 B，也不要 Commit 或 Push。

第二次提示（改良 B）：

改良 A 已完成。請在不破壞現有功能的前提下完成改良 B：「改善多人連線的穩定性和畫面：連不上時 15 秒內顯示原因並建議改用手機熱點；房主暫時斷線時保留房間和代碼並自動重連；玩家區只顯示自己的完整資料，其他人改用淨資產排行」。請保留清楚的 UI 文字、錯誤處理或響應式設計；完成後列出變更清單、潛在副作用與驗證步驟。不要自行 Commit 或 Push。

第三次提示（理解自己的成果）：

請對照這次修改的 Git diff，向我解釋 3 個關鍵改動：每項改動前是什麼狀態、現在如何運作、為何這樣設計。請指出具體檔案與程式位置，並設計 2 題能檢驗我是否理解這次修改的問題。

證據 7： 改良 A 操作截圖、改良 B 操作截圖、至少 1 張 Git diff 或修改檔案清單截圖。

改良 **A**：多人連線







左：首頁分成「一台裝置輪流玩」和「多人連線」兩個入口。中：房主建立房間，拿到房間代碼，輸入名字並選企業。右：另一台裝置用代碼加入，別人選走的企業會變灰。





房主按「開始遊戲」後，每台裝置自動對應自己選的企業（上方顯示「你：小明」）。玩家區只顯示自己的完整資料，其他人只出現在「淨資產排行」。

改良 **B**：連線錯誤處理與畫面精簡





左：連不上房主時，15 秒內顯示原因，並建議改用同一個手機熱點（以前會一直停在「正在連線…」）。右：房主和配對伺服器暫時斷線時，房間和代碼都保留，顯示「暫時離線」並自動重連，已加入的人可以繼續玩。

修改檔案清單（**git diff**）

main 與 feature/my-improvement 的差異（不含自動產生的 game.js）

```
$ git -c core.quotepath=off diff --stat=110 origin/main...origin/feature/my-improvement -- . ':!game.js'
 AGENTS.md                     | 163 ++++++++++++++++++++
 README.md                     |  83 ++++++++++-
 app.js                        | 570 ++++++++++++++++++++
```

任務 8｜測試、檢查變更、修正問題

目標： 確認真正完成，不只是 Codex 說「已完成」。

測試原則：

1. 重新執行專案，測試原本功能仍正常。
2. 操作改良 A，記錄預期與實際結果。
3. 操作改良 B，記錄預期與實際結果。
4. 若有自動化測試、Lint 或 Build 指令，依 README／專案設定執行。
5. 使用 git diff、git status 檢查不相關的變更及敏感檔案。

Codex 提示詞：

請檢查目前未提交的變更。先根據專案設定確認可執行哪些測試、Lint 或 Build 指令；在我確認後執行。請用表格列出「測試項目、操作方法、預期結果、實際結果、通過／失敗」，並指出任何未驗證的部分。不要虛構已執行的測試，也不要自行 Push。

測試	預期結果	實際結果	通過？
原本主要功能	不開房間時照舊在一台電腦輪流玩：拉霸、抽卡、經營、季末結算和多頁籤同步都正常。	單機模式照舊顯示所有玩家卡片和「輪到你」；規則與回合的引擎測試全數通過。	■是 □否
改良 A	不同裝置用代碼加入同一局；每個人在房間裡自己選企業，房主隨時可以開始；開始後每台只能操作自己的企業。	兩支手機在不同網路實機連線遊玩成功。實測發現的問題（手機存著舊局進不了選企業畫面、按拉霸後對局消失）都已修正。瀏覽器實際連線：大廳 19、模式分開 14、座位鎖定與接手 9、完整玩一季 6 項全部通過；組員加入投資與競標後重測，多人競標與投資 20 項也通過。	■是 □否
改良 B	連不上時約 15 秒內顯示原因；房主離線後恢復同一個代碼；玩家區只顯示自己的資料加淨資產排行。	連線逾時 8 個情境、斷線恢復 9＋3 個情境全部通過（修改前的版本分別只過 2 個）；375／1000／1500px 寬度版面正常。	■是 □否
自動化測試／建置（如適用）	node --test 全部通過；node build.mjs 能產生 game.js。	51 個測試全部通過（含組員的投資、競標測試與 100 局完整模擬）；重新建置的 game.js 與提交的版本完全一致。	■是 □否 □不適用

證據 8： 測試畫面輸出；若失敗，附修正前後的錯誤紀錄。

自動化測試與建置

```
$ node --test --test-reporter=spec
✔ 情境商機至少十四種，僅在至少兩位符合資格時開標（3.082485ms）
✔ 每人一次、現金驗證、非適用企業與過期競標皆拒絕（0.831729ms）
✔ 公開快照沒有未揭標金額，非目前回合玩家仍可出價（0.391866ms）
✔ 揭標僅得標者付款，收益生效兩季，其他人完全不扣款（1.602598ms）
✔ 全部放棄不扣款不發效果，同價由可重現亂數決定（0.622851ms）
✔ 成本優惠不超過企業當季成本，效果不憑空增加資產（0.402094ms）
✔ 房間競標訊息只接受明確型別，不接受宣稱自己的玩家身分（0.342945ms）
✔ 負現金但尚未破產者仍能放棄競標，揭標不會卡住（0.493928ms）
✔ 罷工收入歸零也涵蓋商機收入，不會額外領取訂單收益（2.206668ms）
✔ 新對局預設以建立時間作為亂數種子（2.766104ms）
✔ 八位玩家都完成回合後才結算，九位玩家無法開局（40.282702ms）
✔ 撞上罷工幽靈時收入歸零、支出照付，停工一回合且不抽卡（2.759198ms）
✔ 舊存檔載入時補上罷工幽靈狀態（0.852818ms）
✔ 十種企業、十張全球卡、十條規則及基準淨利符合規格（0.448935ms）
✔ E01 依衰減向量計算，且不憑空建立銀行原物料成本（1.62905ms）
✔ E05 依角色調整人力成本；全球卡同步套用所有適用企業（0.551127ms）
✔ 個人、產業與全球事件同季疊加；資產從基準重估而非複利扣減（0.684339ms）
✔ 失業、銀行比率與缺少歷史值的快照狀態正確（4.45365ms）
✔ 事件成本進入通膨公式，海灣戰爭首兩季為 6.4% 與 5.3%（2.005817ms）
✔ 借還款與壞帳操作更新銀行存量，不把借款算成淨資產（0.750322ms）
✔ 聯準會缺值不觸發，F03 優先於 F01，政策下一季生效（1.270615ms）
✔ F07 市場崩盤與 F10 泡沫條件分別走 QE 與升息分支（0.600358ms）
✔ 利率決策在下一季才生效，事件消失不會重設政策利率（2.5233ms）
✔ 兩輪拉霸後才結算全市場一季（4.261277ms）
✔ 棋子逐格路徑與實際落點一致，經過起點只領一次獎金（0.222895ms）
✔ 兩輪 0–6 拉霸的步數範圍為 0–12 格（0.324738ms）
✔ 繞回起點時不顯示事件卡（0.207005ms）
✔ 固定種子可重現拉霸數字與事件（0.434736ms）
{"games":100,"turns":1340,"buys":129,"sells":100,"auctions":251}
✔ 100 個不同亂數種子正常走完整局，投資與競標不破壞結算、存檔及勝負（1015.880588ms）
✔ 買入等額現金轉資產，不憑空增加淨資產（2.135703ms）
✔ 只能自己回合指定格買他人企業，限制現金、張數與每回合一次（0.641382ms）
✔ 賣出按現值收款，別人不可賣，重複賣出被拒絕（0.655289ms）
✔ 實際產業事件影響被投資企業，卡片調價只在抽卡時執行（1.901288ms）
✔ 上下限0到400、混合淨影響決定方向、新買不追溯事件（0.815655ms）
✔ 投資列入季末勝負判定、存檔與多人同步資料（24.222464ms）
✔ 競標期間不能交易，房間訊息驗證型別且不接受自稱玩家（1.865314ms）
✔ 房間代碼為 5 碼且不含易混淆字元（1.468989ms）
✔ 輸入的代碼會轉大寫並略過空白與連字號，格式不符則回傳 null（0.506863ms）
✔ 還有空位時誰都不能操作，房主也不例外（1.418027ms）
✔ 坐滿之後只有坐在目前玩家的裝置能操作（0.291567ms）
✔ 對局結束或尚未開局時沒有人可以操作（0.313477ms）
✔ 認領座位：已被占用會拒絕，坐下後不能自己換位子（0.654598ms）
✔ 房主收回座位後可以重新選，且不會改動原本的物件（0.316258ms）
✔ 淨資產排行由高到低，同分同名次（0.983885ms）
✔ 名字去掉控制字元與前後空白，最多 16 字（3.580166ms）
✔ 大廳：每家企業只能一台裝置選，換企業不改變順序（0.833982ms）
✔ 大廳最多 8 位玩家（0.396593ms）
✔ 開始時依選擇順序建立玩家，補上空白名字並避開重名，座位直接對應（0.430377ms）
✔ 只接受白名單內的訪客訊息，數量必須是有限數字（0.592771ms）
✔ 其他頁籤只採用較新的同場對局修訂版（1.78166ms）
✔ 新對局取代舊對局，舊頁籤不能覆寫新對局（0.548586ms）
ℹ tests 51
ℹ suites 0
ℹ pass 51
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 1123.034384

$ node build.mjs && git status --short
Built game.js (version e814c32a) for direct opening of index.html.
```

51 個自動化測試全部通過（含 100 局完整對局模擬）。最後一行 node build.mjs 之後 git status 沒有任何輸出，代表重新建置的 game.js、index.html 和提交的版本完全相同。

另外用 Playwright 開兩個瀏覽器，以真的 PeerJS 實際連線測試：選企業大廳 19 項、單機與多人分開 14 項、座位鎖定與接手 9 項、斷線恢復 9＋3 項、連線逾時 8 項、房間裡完整玩一季 6 項、多人競標與投資 20 項，全部通過。

修正前後的錯誤紀錄：連線逾時：修改前 8 項只過 2 項（畫面卡在「正在連線…」），修改後 8 項全過。房主斷線：修改前 9 項只過 2 項（房間被拆掉、訪客斷線），修改後 9 項全過。單機和多人分開後，實測發現房主按拉霸對局就消失：修正前的版本在「房間裡完整玩一季」只過 1 項，修正後 6 項全過。

任務 9｜分次 Commit，Push 到自己的 Fork

目標： 讓 GitHub 保留清楚、可追蹤的版本紀錄。

建議： 改良 A 完成並驗證後做第一個 Commit；改良 B 與 README 更新後做第二個 Commit。若已連續完成兩項改良而未 Commit，先請 Codex 分析差異是否能合理拆分，勿捏造不存在的歷程。

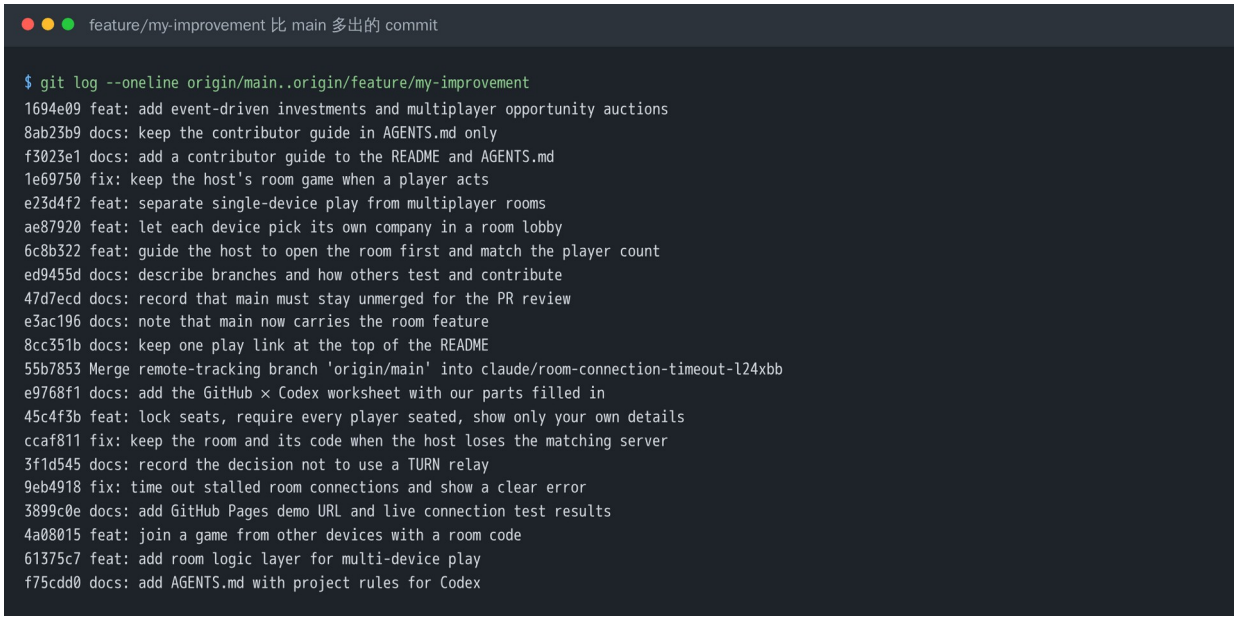
```
git status
git diff
# 先確認沒有 .env、密碼、金鑰或無關檔案，再選擇性加入：
git add README.md path/to/changed-file
git commit -m "feat: add requested interaction"
# 完成另一組修改後，再進行第二次 Commit：
git add README.md path/to/another-file
git commit -m "improve: refine usability and documentation"
# 確認 origin 是自己的 Fork 後：
git push -u origin feature/my-improvement
```

注意： 範例中的 path/to/... 需依專案替換；若實際檔名不同，不可照抄。避免在尚未檢查檔案前直接使用 git add -A。

Codex 提示詞：

> 請先執行或協助我檢查 git status、git diff、git remote -v。整理適合本次 Commit 的檔案與 Commit message，特別檢查有沒有憑證或不應公開的內容。請先提出要執行的 Git 命令並等待確認，確認後才 Commit；Push 前再讓我核對 origin 與分支。

證據 **9**： 學生 GitHub Repository 的 Commits 頁面或 Branch 頁面，至少看得到兩次有意義的 Commit。



feature/my-improvement 比 main 多 21 個 commit，包括多人連線、連線修正、選企業大廳、單機與多人分開，以及組員的企業投資與商機競標（1694e09）。

任務 10｜發表 GitHub Pages

到自己的 Repository → **Settings → Pages**，依專案特性設定發布來源。單純 HTML/CSS/JavaScript 作品，通常可使用 **Deploy from a branch**；需要建置的前端專案，可能需要 GitHub Actions 或專案指定流程。

注意： GitHub Pages 是靜態網站託管，並非通用後端伺服器。不可把密碼、API Key、個資放入公開網站；如範本並不適合 Pages，就以本機執行成果影片或截圖驗收。

展示網址：https://soymilk0211.github.io/bigdata-week7/

請 AI 協助判斷 本次作業相對原始範本，增加了 約 60 % 的功能或改進

AI 說明：以原始範本（main 分支 f66c429）為基準，不含自動產生的 game.js，程式與測試從約 150 KB 增加到約 240 KB（+60%），測試從 21 個增加到 51 個。原本的遊戲規則沒有改，增加的是：① 和單機分開的「多人連線」模式（房間代碼與邀請連結、WebRTC 點對點同步、進房自己選企業的大廳、座位鎖定與接手、連線逾時與房主斷線恢復、只看自己資料的淨資產排行），約 +35%；② 組員 Shuoting93 加入的企業投資卡與 14 種商機競標，約 +25%。綜合程式量與功能，判斷約增加 60%。

附上 AI 說明的截圖：說明全文如上，由 Claude Code 依 git 統計整理，可用 git diff --stat main...feature/my-improvement 重現。

口頭展示（每組 2～3 分鐘）

依序說明：**原作品的用途→ 改良 A→ 改良 B**。請實際操作至少一項改良，不只展示程式碼或 AI 對話。

學習反思（寫在紙上）

1. 你最初以為 Fork、Clone、Commit、Push 分別在做什麼？實作後有何修正？
2. Codex 完成了哪些工作？有哪些功能、錯誤或設計需要你自己判斷？
3. 這個作品下一步還可以增加什麼功能？為什麼？
4. 自評本次作業相對於原始範本，增加了多少(%)改良

作答：

1. 一開始我以為 Fork 和 Clone 都只是「複製一份專案」，Commit 就是存檔、Push 就是上傳。實作後才分清楚：Fork 是在 GitHub 上把老師的範本複製到自己的帳號；Clone 是把自己的 Fork 下載到電腦，讓 Codex 修改；Commit 是在本機留下一筆有說明的版本，Push 才會送到 GitHub。我也學到 Pull Request 是用來比較兩條分支的差異：我們一度把功能分支合併進 main，後來發現作業要用 PR 看 main 和 feature/my-improvement 的差異，又把 main 退回原本的版本。

2. Codex 協助我讀懂原本的程式、寫 AGENTS.md，做出房間代碼和用代碼加入的第一版；後續再用 Claude Code 修正連線逾時和房主斷線，做出座位規則、選企業大廳、單機與多人分開，並寫測試、更新文件。需要我自己判斷的有：要不要加 TURN 中繼（需要把金鑰放在公開網頁，我決定不加）、座位規則（一人一台、選了就鎖定、全員入座才開始）、改成進房自己選企業、單機和多人要分開、畫面只顯示自己的資料和淨資產排行（和組員討論後決定）。很多問題是我用手機實測才發現的，例如同一支手機開兩個分頁連不上、手機存著舊局時進不了選企業畫面、按拉霸後對局消失，AI 的自動測試一開始都沒有抓到。

3. 下一步可以做：① 淘汰玩家的併吞規則（組員已經做了投資和商機競標，併吞還沒做）；② 玩家中途退出，讓他的公司改成無人經營，不用一直等人接手；③ 房主重新整理後能恢復房間和對局，目前房主一離開房間就結束了；④ 把瀏覽器自動測試放進 Repo，組員改程式後也能自己跑。原因是實際在課堂上玩時，最常遇到的是有人斷線或離開，這些都會讓整局卡住。

4. 自評約 60%：原作品只能在一台裝置輪流玩，現在多了每人一台手機的多人連線模式（含選企業大廳、斷線處理、只看自己資料的排行），加上組員做的企業投資與商機競標；原本的遊戲規則沒有改。程式量從約 150 KB 增加到約 240 KB，測試從 21 個增加到 51 個。